

# Transformers

---

---

# Introduction and Overview

---

---

# Transformers

Google Scholar

attention is all you need

Articles About 85,60,000 results (0.04 sec)

Any time

Since 2024

Since 2023

Since 2020

Custom range...

**Attention is all you need**

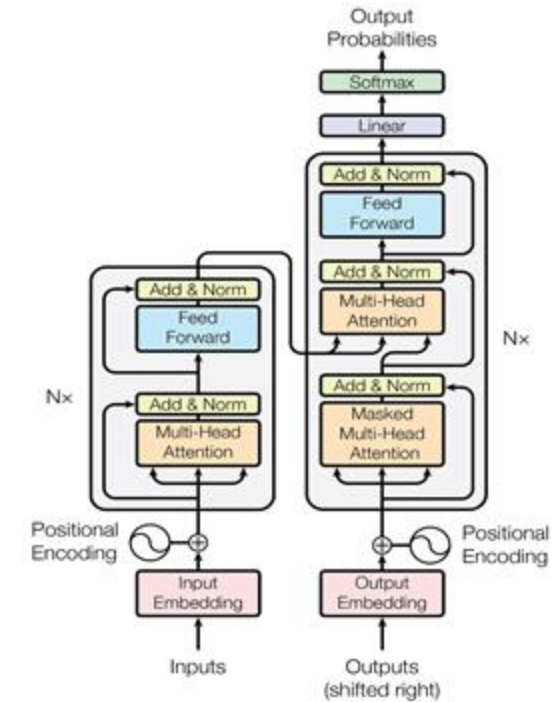
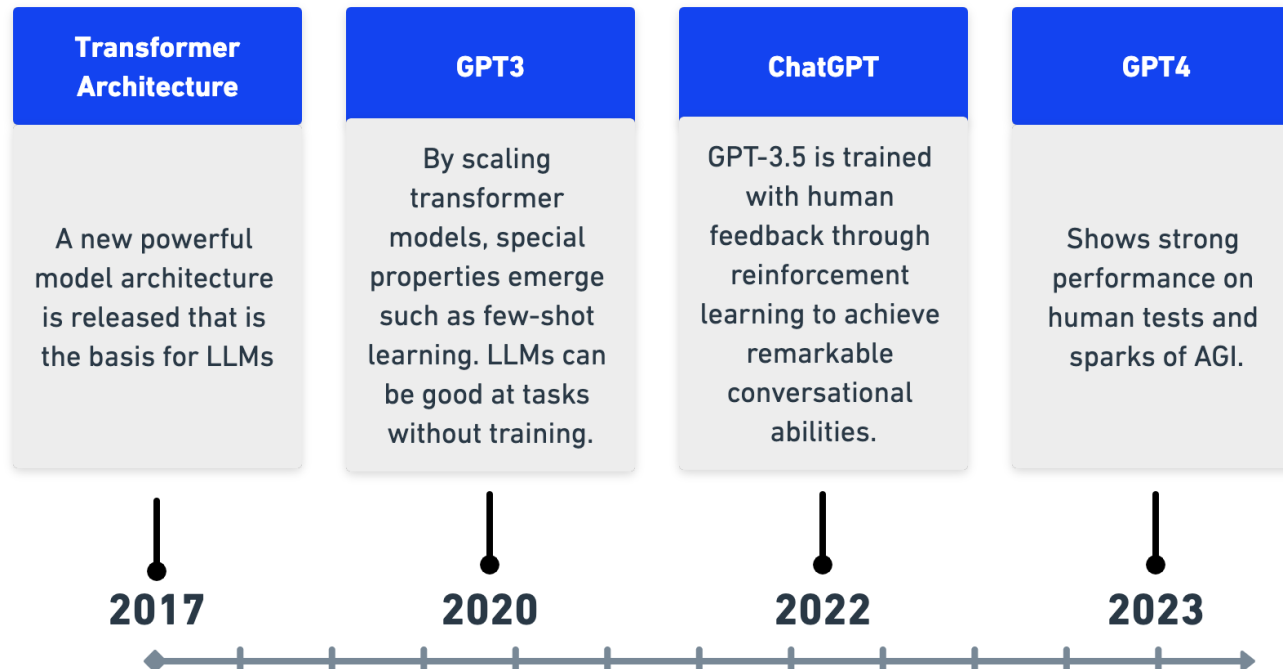
A Vaswani, N Shazeer, N Parmar... - Advances in neural ..., 2017 - proceedings.neurips.cc

... to attend to **all** positions in the decoder up to and including that position. **We need** to prevent ... **We** implement this inside of scaled dot-product **attention** by masking out (setting to  $-\infty$ ) ...

☆ Save Cite Cited by 123714 Related articles All 91 versions

[PDF] neurips.cc

Task: Translate from one language to another



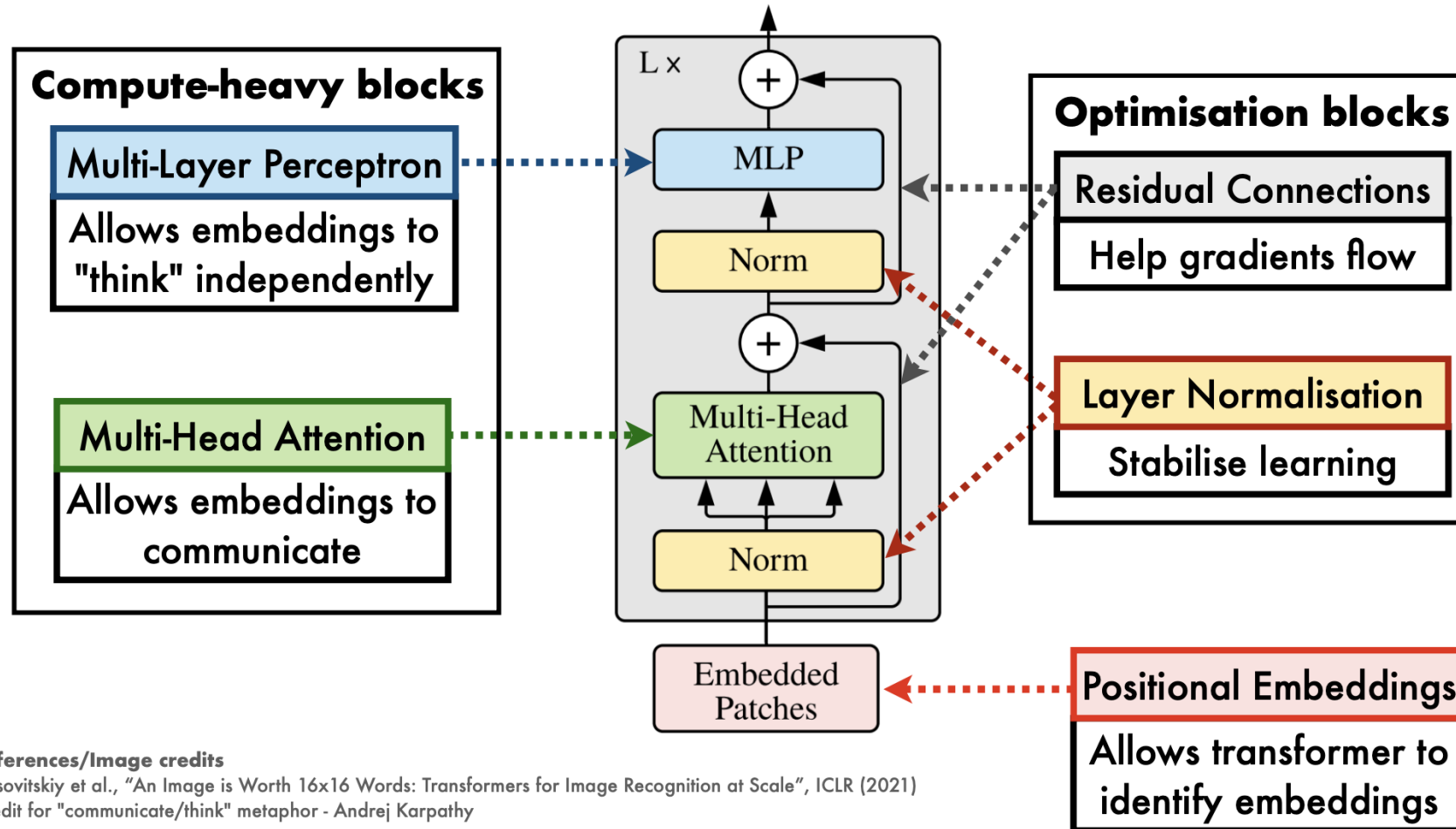
Transformer Encoder  
Transformer Decoder

Attention is all you need (2017)

# Transformers

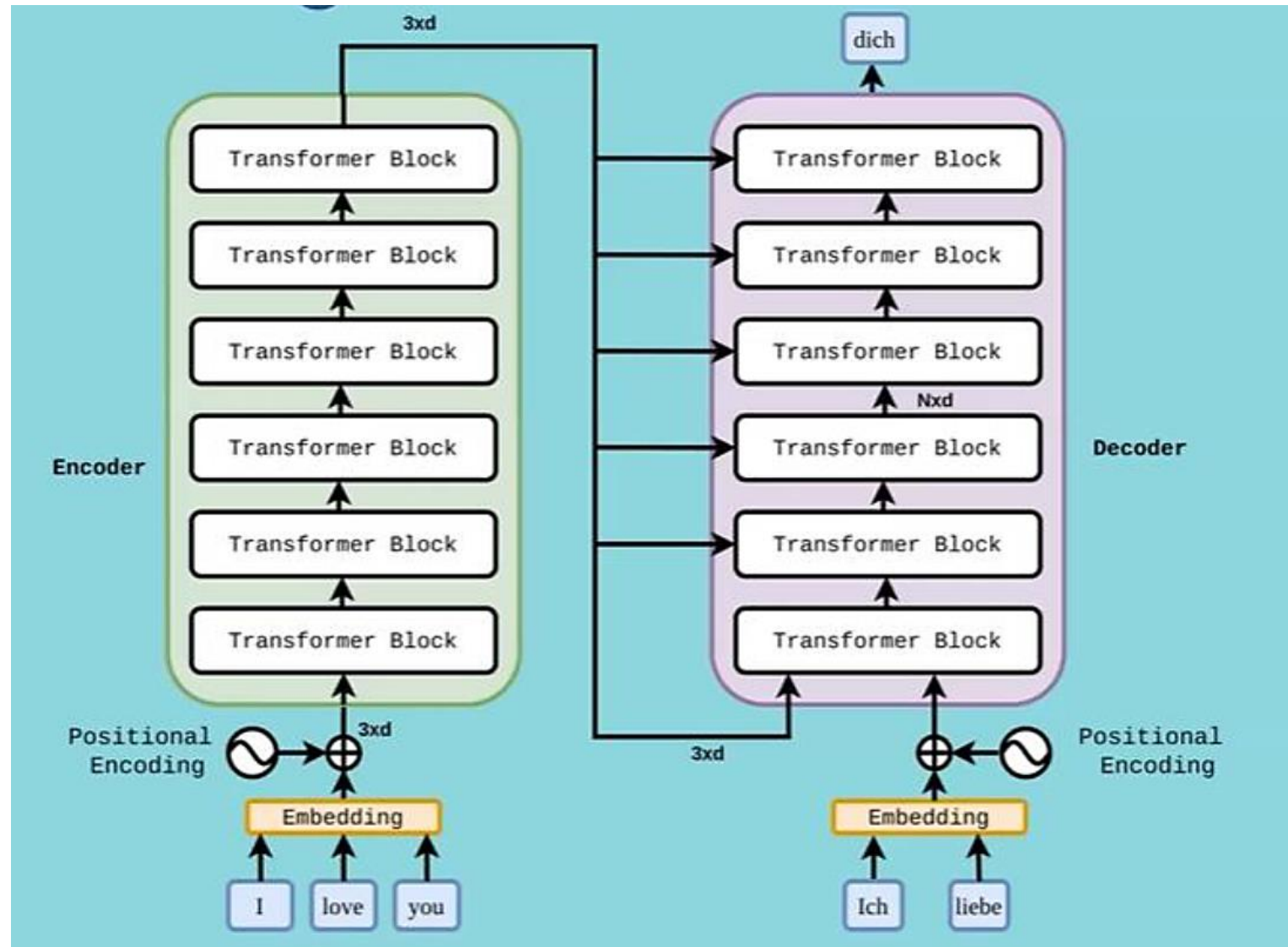
## Transformer Encoder

Five key ideas



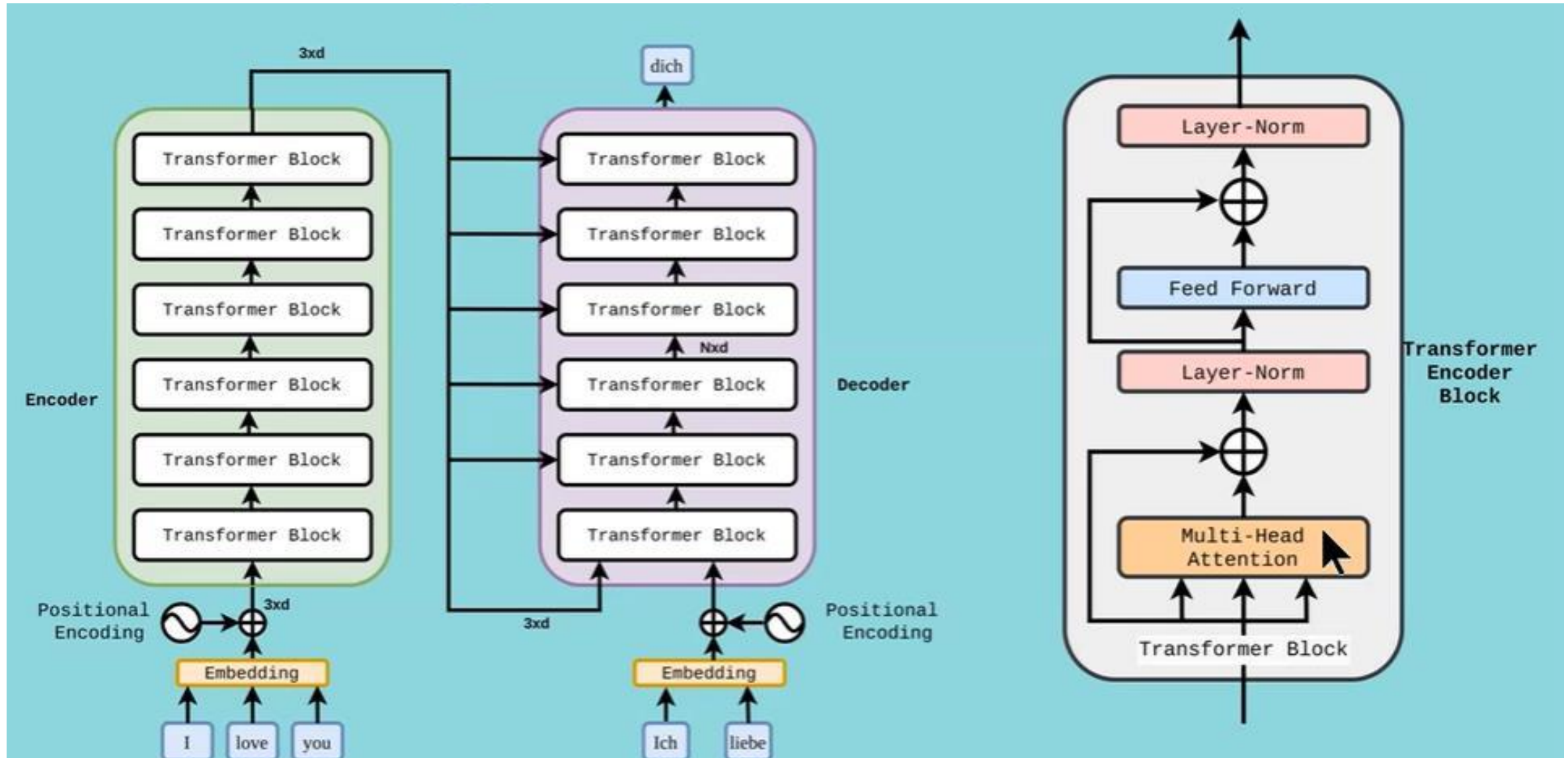
# Transformers

Good for  
Sets and  
Sequences



Encoder only  
Decoder only  
Encoder-  
Decode

# Transformer

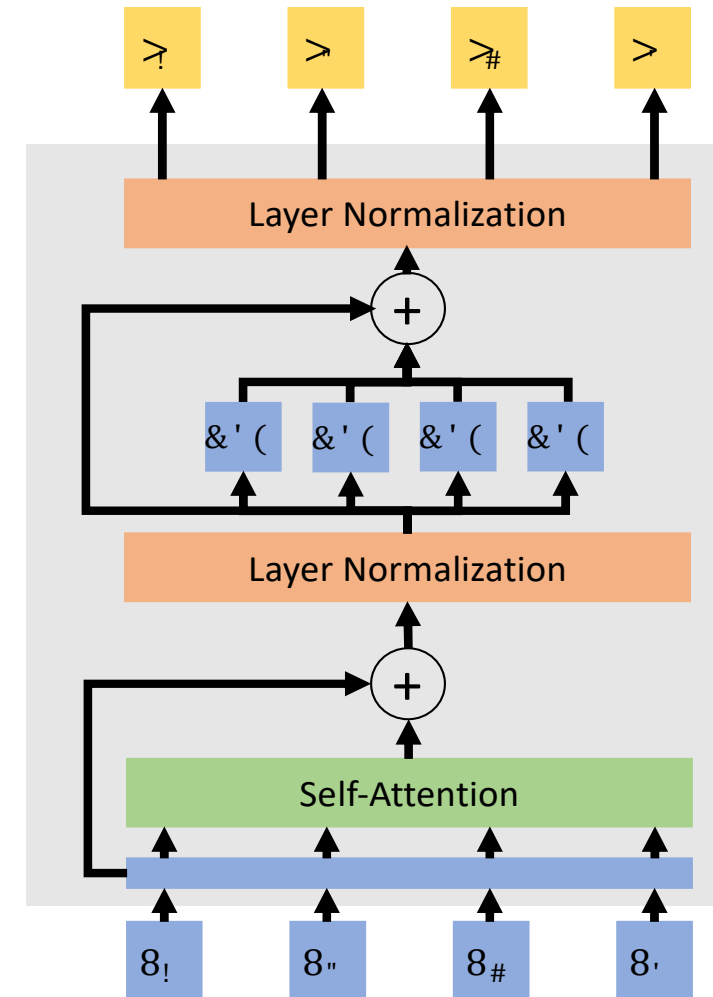


# Blank Slide

# Building Block

## Transformer Block:

- **Input:** Set of vectors  $x$
- **Output:** Set of vectors  $y$
- Self-attention is the only interaction between vectors!
- Layer norm and MLP work independently per vector
- Highly scalable, highly parallelizable





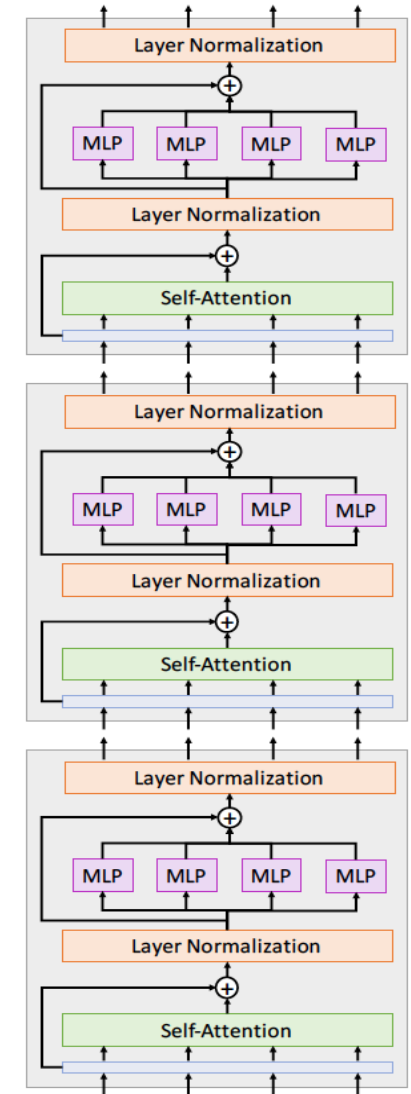
# Transformer

## Transformer Block:

- **Input:** Set of vectors  $x$
- **Output:** Set of vectors  $y$
- Self-attention is the only interaction between vectors!
- Layer norm and MLP work independently per vector
- Highly scalable, highly parallelizable

A **Transformer** is a sequence of transformer blocks

**Vaswani et al:**  
12 blocks,  $D_v = 512$ , 6 heads



# Transformer

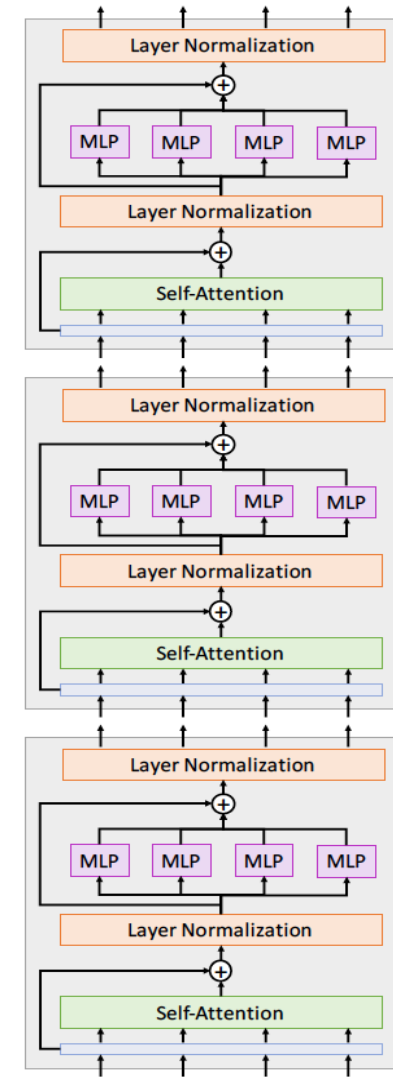
“ImageNet Moment for Natural Language Processing”

## Pretraining:

- Download a lot of text from the internet
- Train a giant Transformer model for language modeling

## Finetuning:

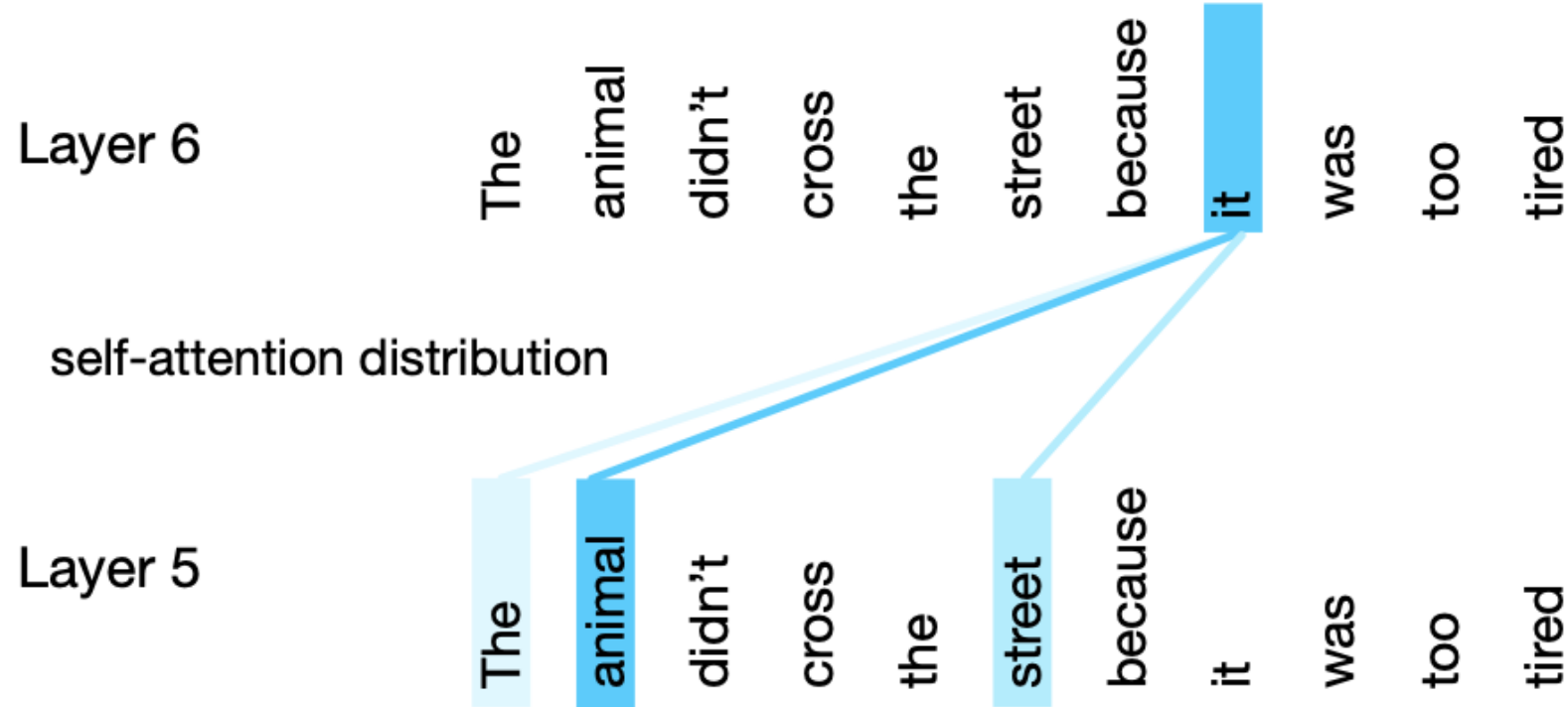
- Fine-tune the Transformer on your own NLP task



# Blank Slide

# Recap of Self Attention

# Self Attention



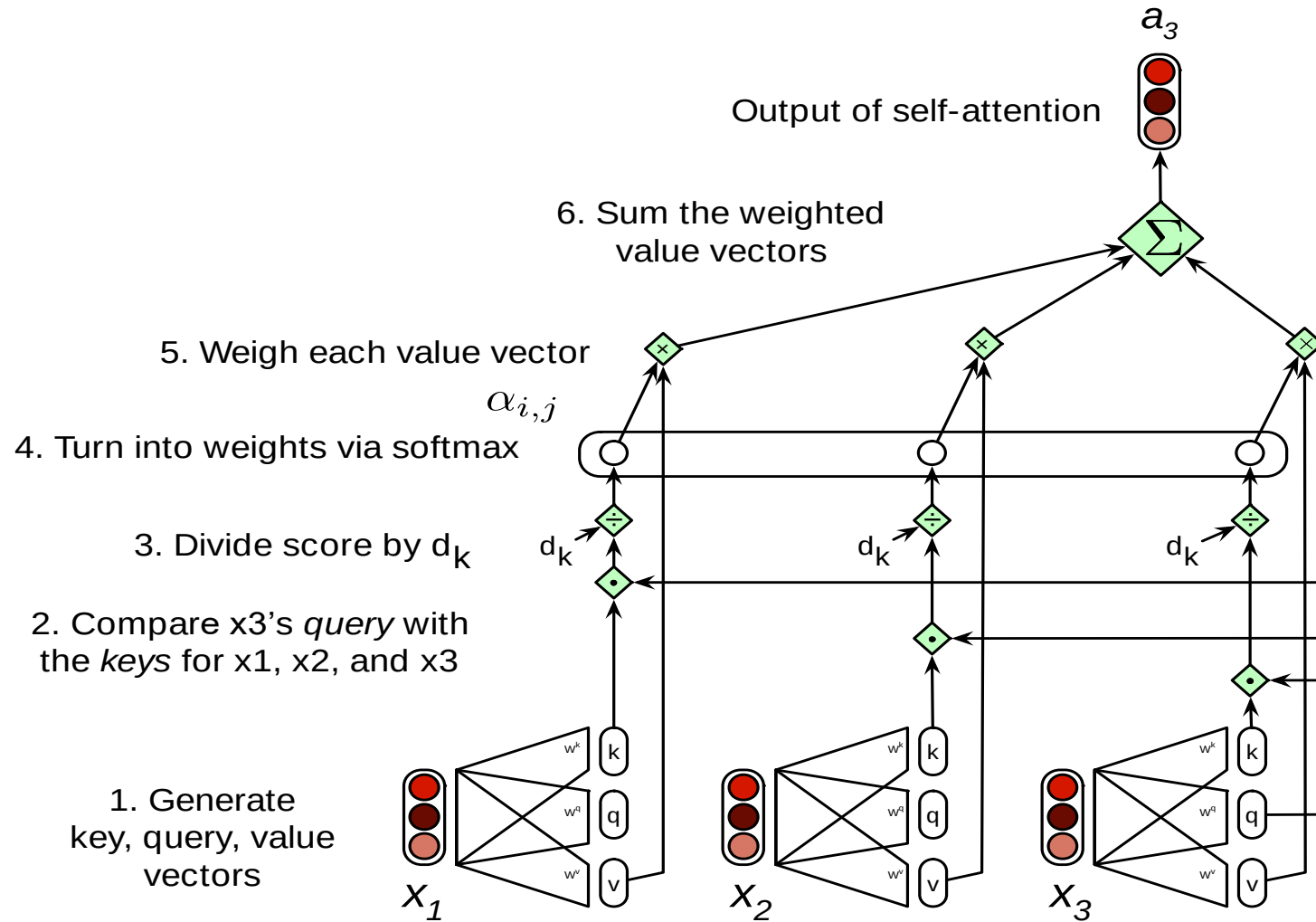
# Query, Key and Value

- As *the current focus of attention* when being compared to all of the other preceding inputs. We'll refer to this role as a **query**.
- In its role as *a preceding input* being compared to the current focus of attention. We'll refer to this role as a **key**.
- And finally, as a **value** used to compute the output for the current focus of attention.

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

# Blank Slide

# Example Computation





# Computation

$$q_i = x_i W^Q; k_i = x_i W^K; v_i = x_i W^V$$

$$\text{score}(x_i, x_j) = \frac{q_i \cdot k_j}{d_k}$$

$$a_{ij} = \text{softmax}(\text{score}(x_i, x_j))$$

$$a_i = \sum_{j \leq i} a_{ij} v_j$$

# Blank Slide

# Parallelization of Computation

$$\mathbf{Q} = \mathbf{XW}^{\mathbf{Q}}; \quad \mathbf{K} = \mathbf{XW}^{\mathbf{K}}; \quad \mathbf{V} = \mathbf{XW}^{\mathbf{V}}$$

$$\mathbf{A} = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left( \frac{\mathbf{QK}^{\top}}{\sqrt{d_k}} \right) \mathbf{V}$$

# Blank Slide

---

---

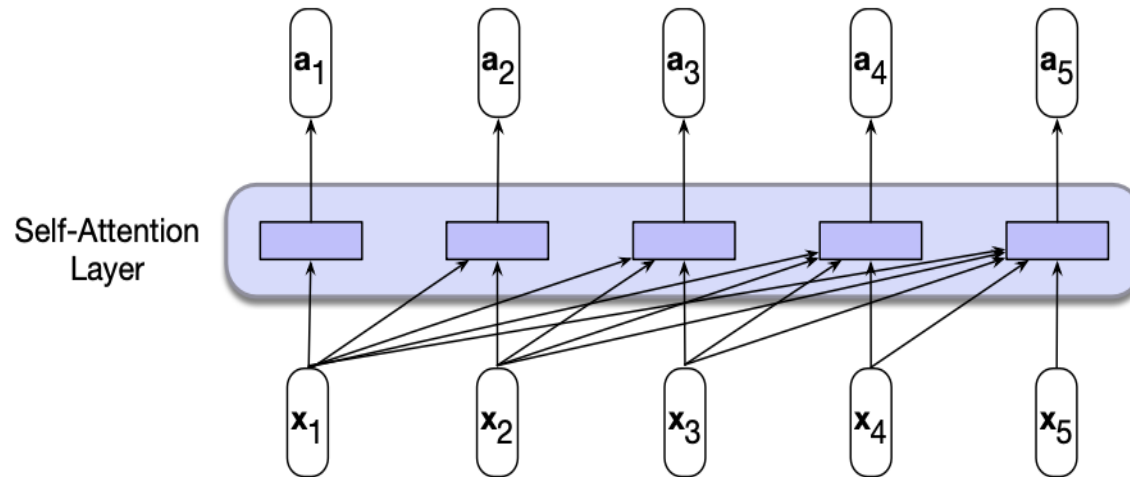
# Masked and Multi-Head Attention

---

---



# Self Attention: Causal/masked



Unlike RNNs, computations at each step are independent of all other time steps and can be done parallelly.

Masking out the future

$$QK^T$$

N

|               |               |               |               |               |
|---------------|---------------|---------------|---------------|---------------|
| $q1 \cdot k1$ | $-\infty$     | $-\infty$     | $-\infty$     | $-\infty$     |
| $q2 \cdot k1$ | $q2 \cdot k2$ | $-\infty$     | $-\infty$     | $-\infty$     |
| $q3 \cdot k1$ | $q3 \cdot k2$ | $q3 \cdot k3$ | $-\infty$     | $-\infty$     |
| $q4 \cdot k1$ | $q4 \cdot k2$ | $q4 \cdot k3$ | $q4 \cdot k4$ | $-\infty$     |
| $q5 \cdot k1$ | $q5 \cdot k2$ | $q5 \cdot k3$ | $q5 \cdot k4$ | $q5 \cdot k5$ |

N

What is the typical value of N?  
4096 tokens

# Blank Slide

# Masked Attention

Don't let vectors "look ahead" in the sequence

## Inputs:

Input vectors:  $X$  (Shape:  $N_X \times D_Q$ )

Key matrix:  $W_K$  (Shape:  $D_X \times D_Q$ )

Value matrix:  $W_V$  (Shape:  $D_X \times D_V$ )

Query matrix:  $W_Q$  (Shape:  $D_Q \times D_Q$ )

## Computation:

Query Vectors  $Q = XW_Q$

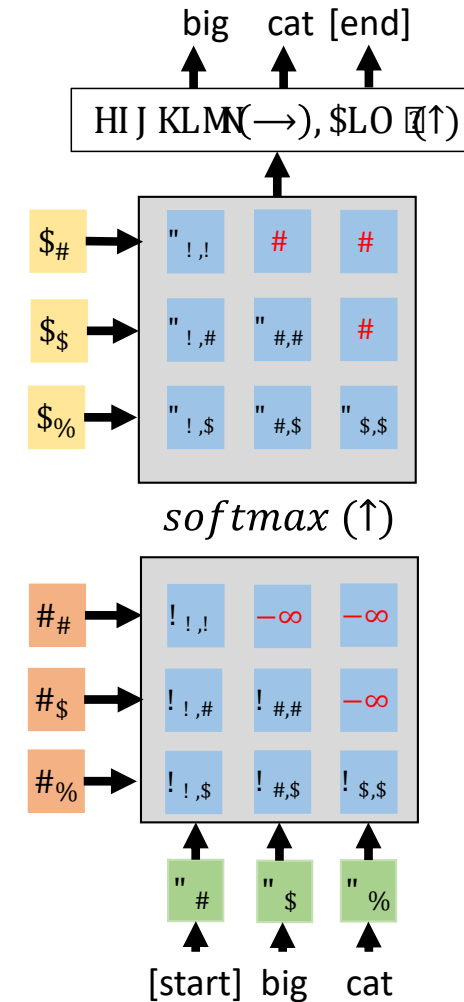
Key vectors:  $K = XW_K$  (Shape:  $N_n \times D_k$ )

Value Vectors:  $V = XW_V$  (Shape:  $N_n \times D_v$ )

Similarities:  $E = \frac{QK^T}{\sqrt{D_k}}$  (Shape:  $N_n \times N_n$ )  $E_{\%} = (Q_{\%} \cdot K^T) / \sqrt{D_k}$

Attention weights:  $A = \text{softmax}(E, \text{dim} = 1)$  (Shape:  $N_n \times N_n$ )

Output vectors:  $Y = AV$  (Shape:  $N_n \times D_v$ )  $Y_{\%} = \sum A_{\%} V$

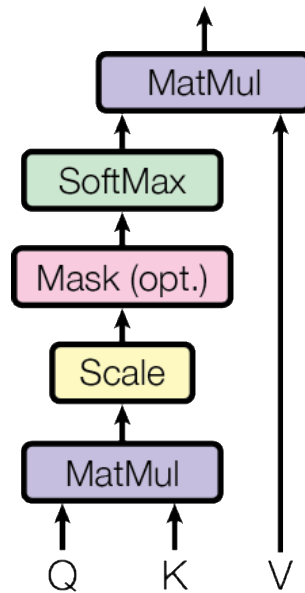




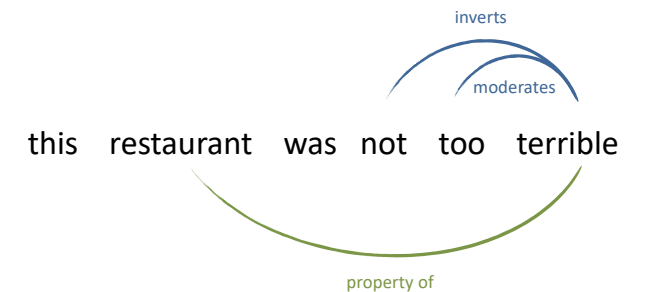
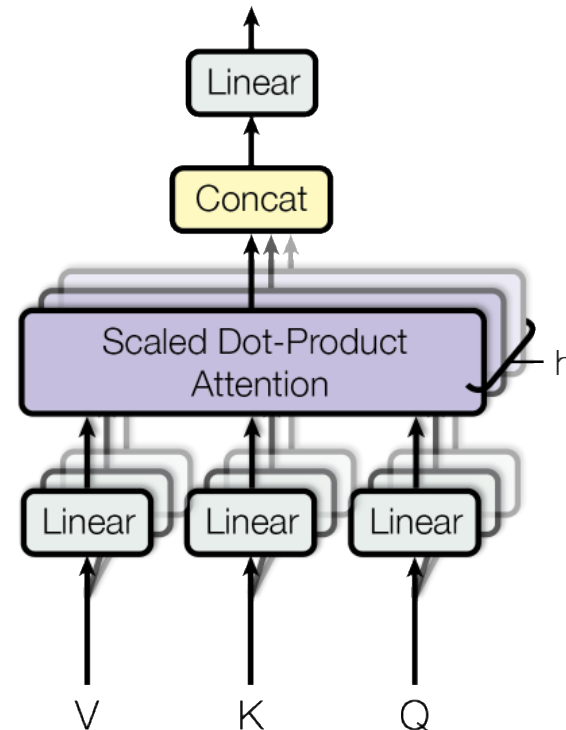
# Multi Head Attention: Why?

- Why are we limited to one query, key and value?
- Multiple Feature Maps (like Convolution Layers)

Scaled Dot-Product Attention



Multi-Head Attention



# Masked Multi-Head Attention

- Masked multi-head attention: multi-head where some values are masked (i.e., probabilities of masked values are nullified to prevent them from being selected).
- When decoding, an output value should only depend on previous outputs (not future outputs). Hence we mask future outputs.

$$attention(Q, K, V) = softmax\left(\frac{Q^T K}{\sqrt{d_k}}\right) V$$

$$maskedAttention(Q, K, V) = softmax\left(\frac{Q^T K + M}{\sqrt{d_k}}\right) V$$

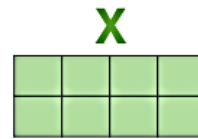
where  $M$  is a mask matrix of 0's and  $-\infty$ 's

# Blank Slide

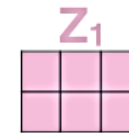
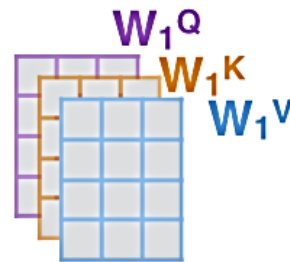
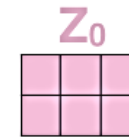
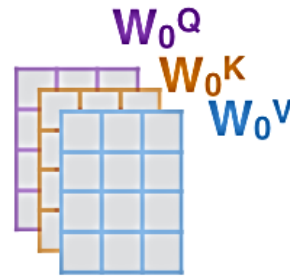
# Multi-Head Attention

- 1) This is our input sentence\*
- 2) We embed each word\*
- 3) Split into 8 heads. We multiply  $X$  or  $R$  with weight matrices
- 4) Calculate attention using the resulting  $Q/K/V$  matrices
- 5) Concatenate the resulting  $Z$  matrices, then multiply with weight matrix  $W^O$  to produce the output of the layer

Thinking  
Machines



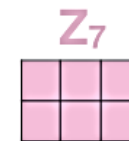
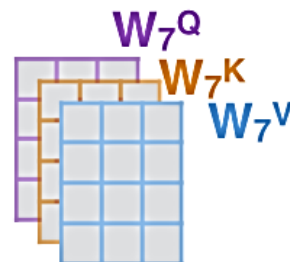
\* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



...

...

...



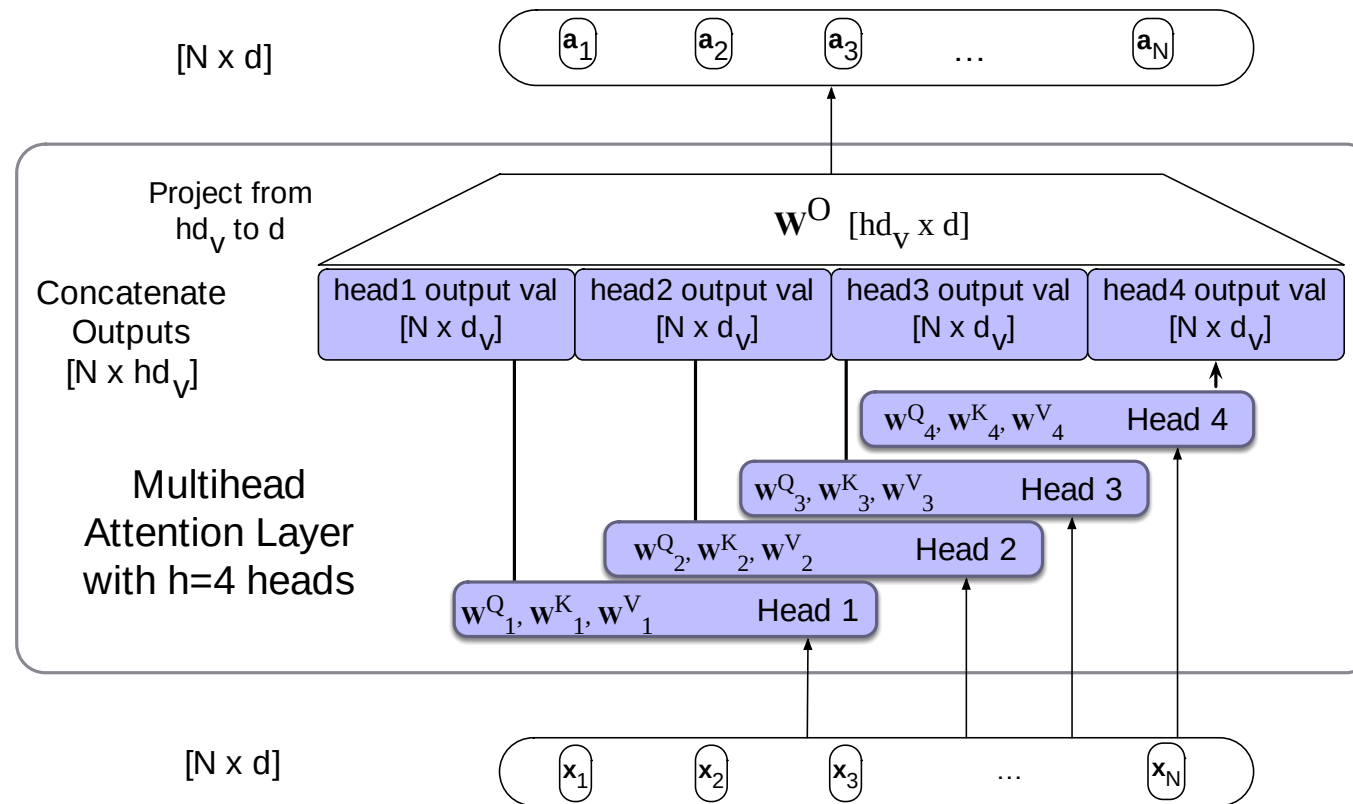
$W^O$



$Z$



# Multi Head Attention



$$\mathbf{Q} = \mathbf{XW}_i^Q ; \mathbf{K} = \mathbf{XW}_i^K ; \mathbf{V} = \mathbf{XW}_i^V$$

$$\text{head}_i = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$$

$$\mathbf{A} = \text{MultiHeadAttention}(\mathbf{X}) = (\text{head}_1 \oplus \text{head}_2 \dots \oplus \text{head}_h) \mathbf{W}^O$$

# Blank Slide

---

---

# Others in the Transformer Blocks

---

---

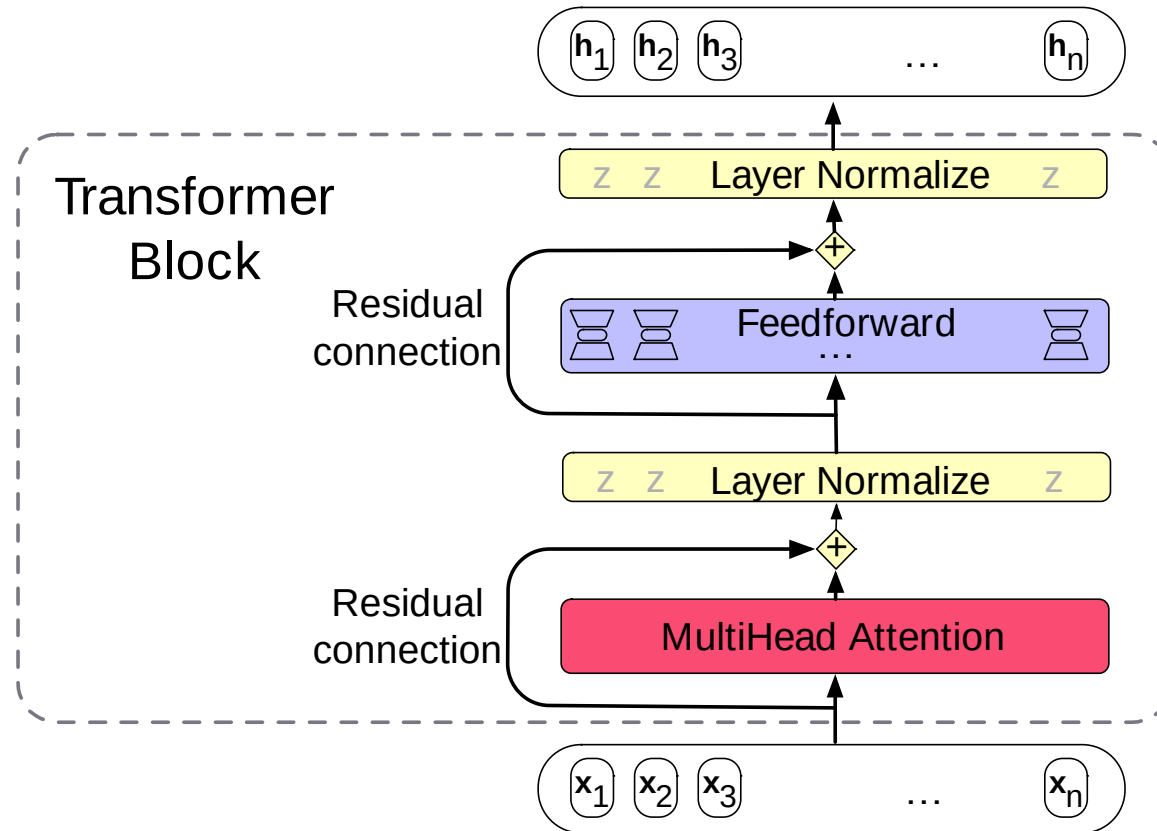
—

—

---

---

# Transformer Block



$$\mathbf{T}^1 = \text{SelfAttention}(\mathbf{X})$$

$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1$$

$$\mathbf{T}_3 = \text{LayerNorm}(\mathbf{T}^2)$$

$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3)$$

$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3$$

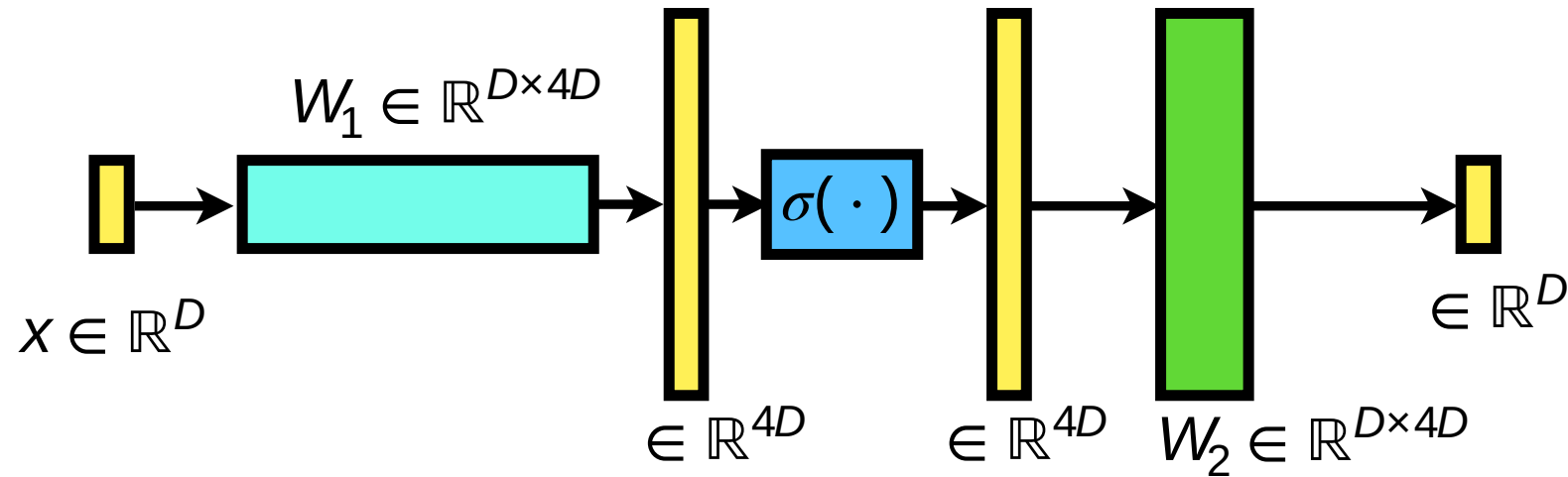
$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5)$$

Both X and H are of  $N \times d$



# Blank Slide

# MLP/Feed Forward Layer



$$\text{MLP}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

# Layer Norm Enhances the Stability

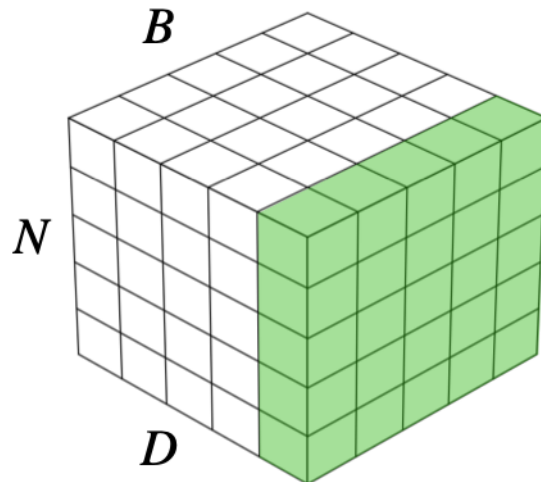
$$\mu = \frac{1}{d_h} \sum_{i=1}^{d_h} x_i$$

$$\sigma = \sqrt{\frac{1}{d_h} \sum_{i=1}^{d_h} (x_i - \mu)^2}$$

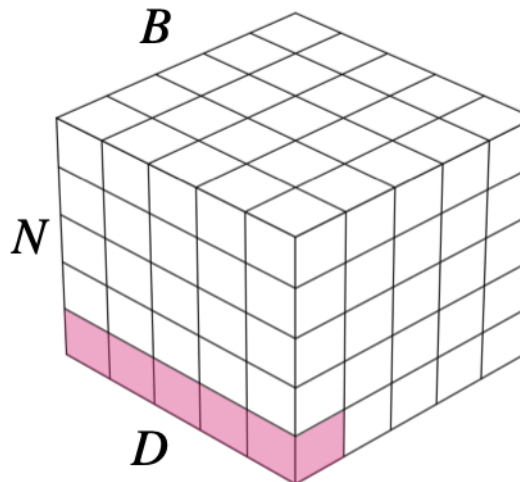
$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

$$\text{LayerNorm} = \gamma \hat{\mathbf{x}} + \beta$$

BatchNorm



LayerNorm



LayerNorm has

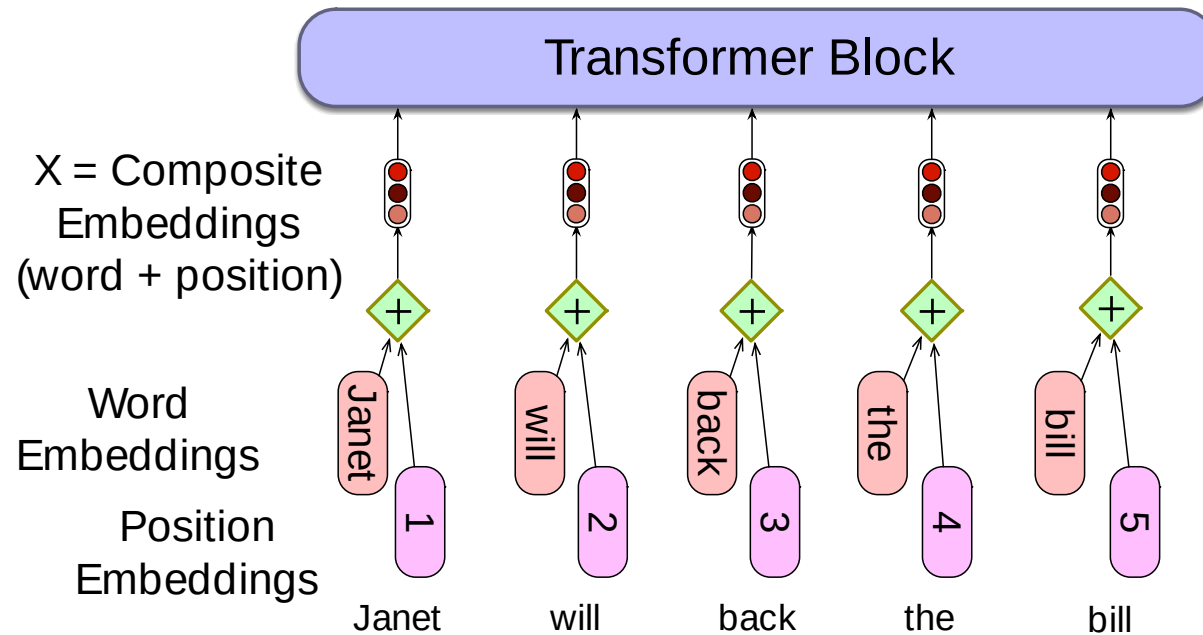
- No dependence on batch dim.
- Same procedure at train/test time

# Blank Slide

# Positional Encoding

- Sets vs Sequences=> Strategy: Introduce “Position”
- Position embedding soon after input embedding
- “Bag of words” to “ordered words”

# Position Embedding

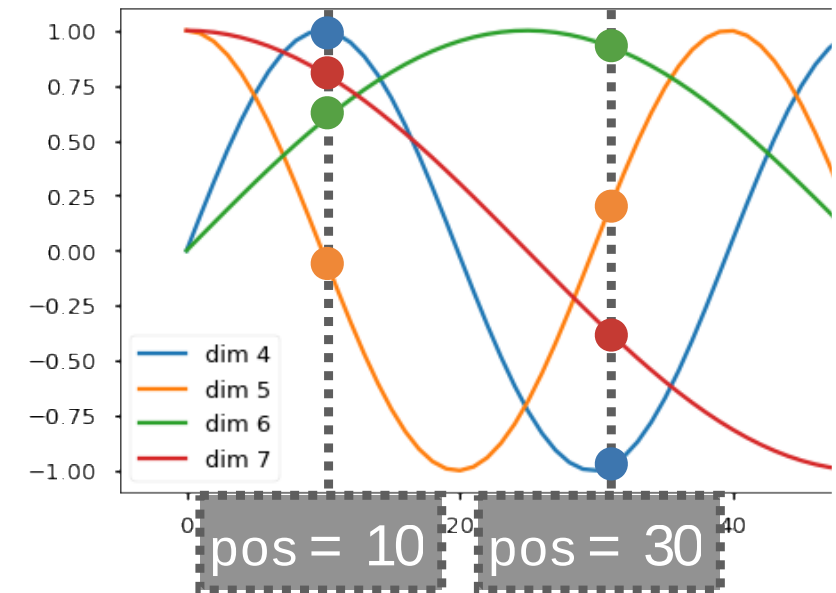


Absolute Positions

Choose static functions (say sinusoids) that map integer inputs to real valued vectors that captures inherent relationship between vectors

$$PE(pos, 2i) = \sin\left(\frac{pos}{10,000^{2i/D}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10,000^{2i/D}}\right)$$



# Blank Slide

# With Positional Encoding

## Inputs:

Input vectors:  $X$  (Shape:  $N_X \times D_Q$ )

Key matrix:  $W_K$  (Shape:  $D_X \times D_Q$ )

Value matrix:  $W_V$  (Shape:  $D_X \times D_V$ )

Query matrix:  $W_Q$  (Shape:  $D_Q \times D_Q$ )

## Computation:

Query Vectors  $Q = XW_Q$

Key vectors:  $K = XW_K$  (Shape:  $N_X \times D_Q$ )

Value Vectors:  $V = XW_V$  (Shape:  $N_X \times D_V$ )

Similarities:  $E = \frac{QX^T}{\sqrt{D_Q}}$  (Shape:  $N_X \times N_X$ )  $E_{i,j} = (Q_i \cdot K_j) / \sqrt{D_Q}$

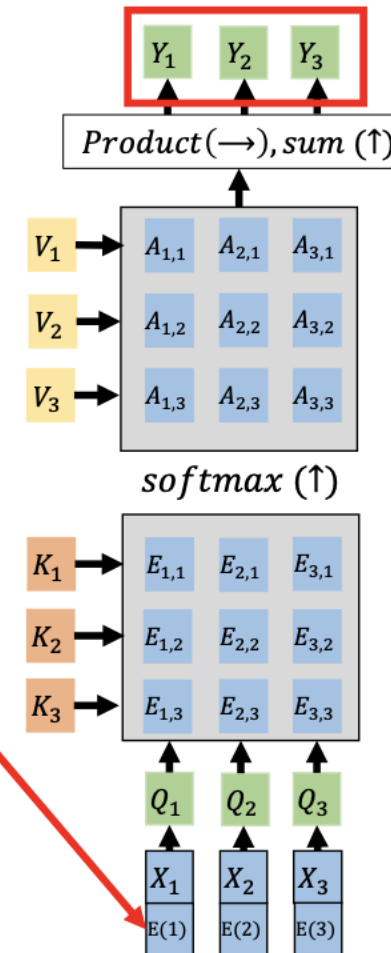
Attention weights:  $A = \text{softmax}(E, \text{dim} = 1)$  (Shape:  $N_X \times N_X$ )

Output vectors:  $Y = AV$  (Shape:  $N_X \times D_V$ )  $Y_i = \sum_j A_{i,j} V_j$

Self attention doesn't "know" the order of the vectors it is processing!

In order to make processing position aware, concatenate or add **positional encoding** to the input

$E$  can be learned lookup table, or fixed function





---

# Architecture: Summary

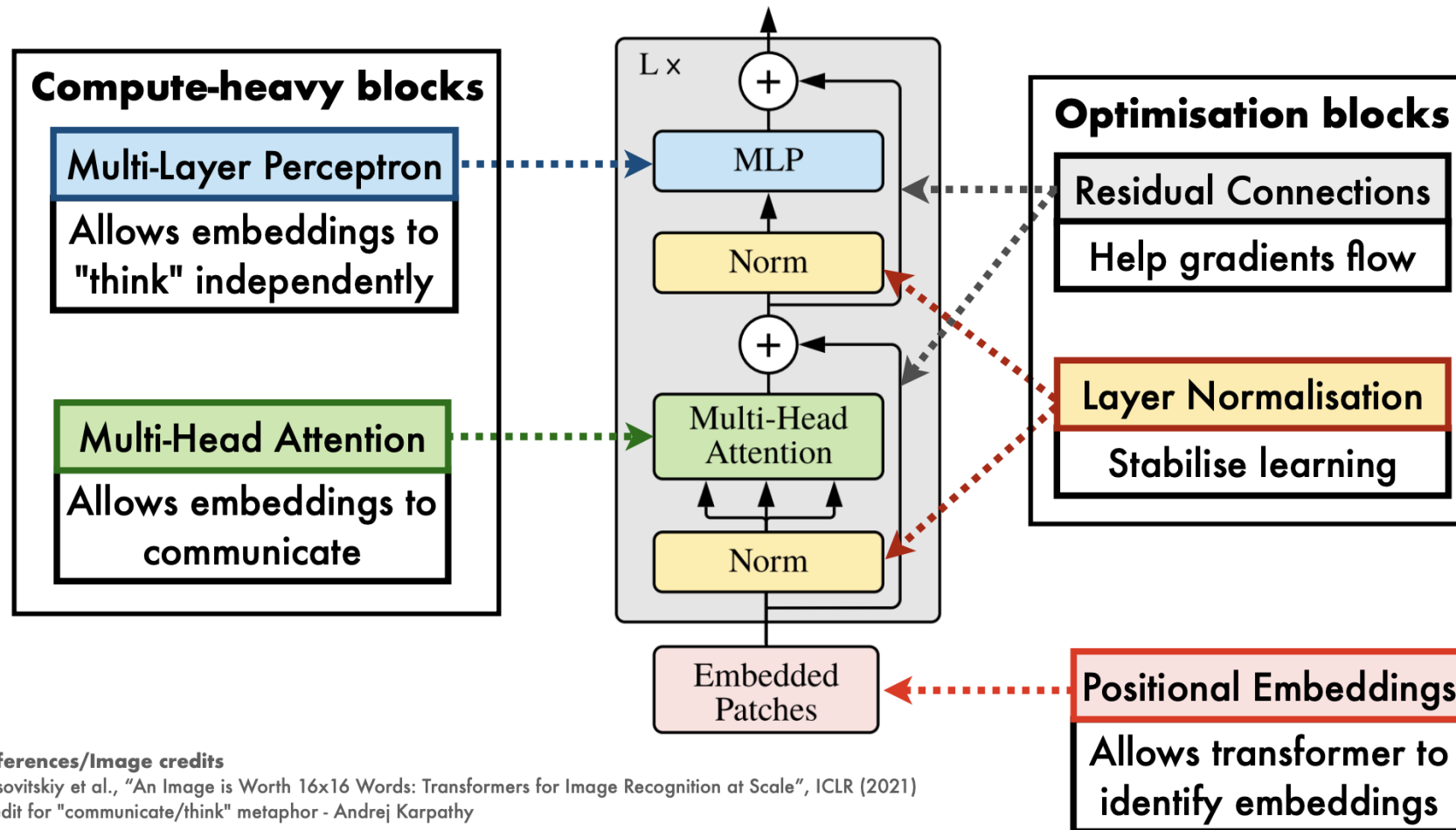
---



# Transformers

## Transformer Encoder

Five key ideas



# Transformers: Summary

- Transformers can attend complete sequence and thereby handle long range dependency.
- Transformers
  - Effective use of attention
  - Implementation that can parallelize
  - Scalability to high complexity models and large datasets
- Key Ideas in Transformers
  - Self Attention
  - Multi Headed Attention
  - Positional Encoding
  - Encoder Decoder Architecture

# Thanks!!

## Questions?